

Calcolatori Elettronici

5 febbraio 2024

Teoria

Esercizio 1 (12 punti): Data una sequenza infinita di caratteri in ingresso appartenenti all'alfabeto "a, i, z", progettare la rete sequenziale che riconosca le occorrenze della sequenza "zia", ipotizzando che l'alfabeto di uscita sia "sequenza riconosciuta, sequenza non riconosciuta". La rete deve essere in grado di riconoscere anche sequenze sovrapposte. Realizzare il circuito equivalente dell'equazione minimizzata di eccitazione di un singolo flip flop di stato.

Esercizio 2 (12 punti): Disegnare l'architettura interna dello z64 e scrivere il microprogramma per supportare l'esecuzione dell'istruzione `cmpb $1, %al`, considerando anche la fase di fetch. Indicare anche i segnali di controllo associati a ciascuna microoperazione.

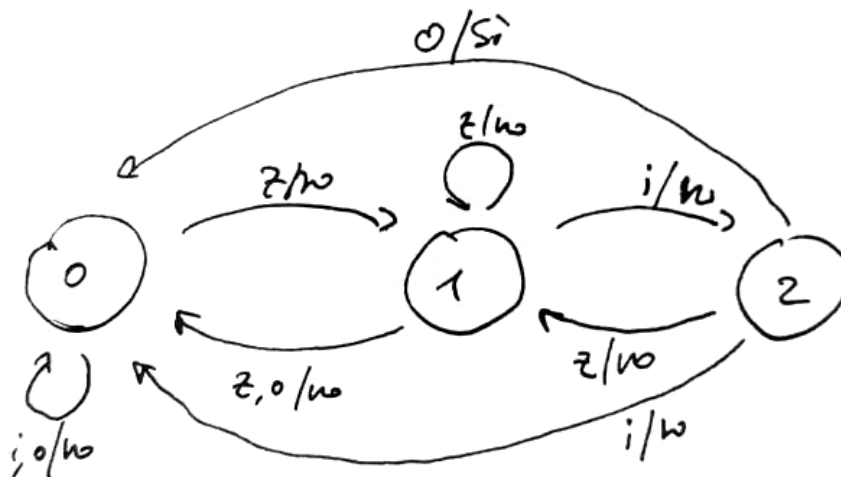
Esercizio 3 (6 punti): Si considerino i numeri $A=1X2$ e $B=3Y4$, dove X e Y sono rispettivamente il penultimo e l'ultimo numero della propria matricola. Convertire entrambi i numeri in complemento a due a 16 bit e calcolare $A+B$ mostrando tutti i passaggi. Esprimere il risultato della somma anche in codifica esadecimale.

Soluzioni

Esercizio 1

Gli alfabeti di input e di output vengono codificati con: $I = \{a = 00, i = 01, z = 10\}$, $O = \{\text{sequenza riconosciuta} = 1, \text{sequenza non riconosciuta} = 0\}$.

L'automa in grado di riconoscere la stringa indicata, realizzato come macchina di Mealy, è il seguente:



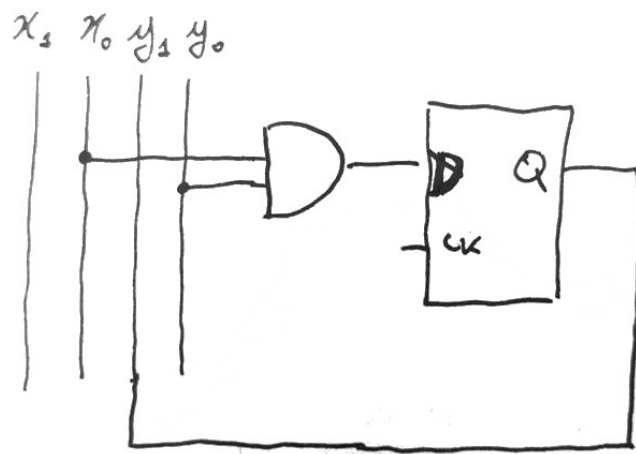
Gli stati vengono codificati con la loro rappresentazione numerica in base 2. La tabella degli stati e transizioni è la seguente:

x_1	x_0	y_1	y_0	y'_1	y'_0	z
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	1
0	0	1	1	—	—	—
0	1	0	0	0	0	0
0	1	0	1	1	0	0
0	1	1	0	0	0	0
0	1	1	1	—	—	—
1	0	0	0	0	1	0
1	0	0	1	0	1	0
1	0	1	0	0	1	0
1	0	1	1	—	—	—
1	1	0	0	—	—	—
1	1	0	1	—	—	—
1	1	1	0	—	—	—
1	1	1	1	—	—	—

Le funzioni booleane derivanti da questa tabella hanno molte don't care condition. Volendo minimizzare la funzione booleana che calcola y'_1 , si può procedere utilizzando una mappa di Karnaugh come segue:

$x_1 x_0$	00	01	11	10
$y_1 y_0$ 00	0	0	—	0
01	0	1	—	0
11	—	—	—	—
10	0	0	—	0

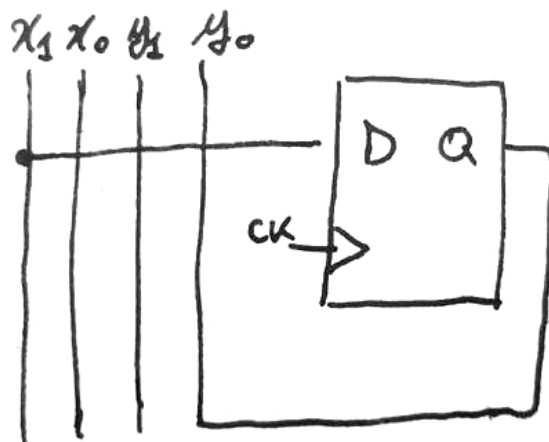
La funzione minimizzata è quindi $y'_1 = x_0 y_0$, che porta al circuito equivalente:



Se avessimo invece voluto minimizzare y'_0 avremmo ottenuto:

$x_1 \ x_0$	00	01	11	10
$y_1 \ y_0$				
00	0	0	-	1
01	0	0	-	1
11	-	-	-	-
10	0	0	-	1

da cui si ricava l'equazione di eccitazione $y'_0 = x_1$, il cui circuito equivalente è:



Esercizio 2

Le microoperazioni per l'istruzione sono:

1. $MAR \leftarrow RIP$
2. $MDR \leftarrow (MAR); RIP \leftarrow RIP + 8$
3. $IR \leftarrow MDR$
4. TEMP1 \gets RAX\$
5. $TEMP2 \leftarrow IR[0 : 31]$
6. $ALU_OUT[SUB]$

L'ultima microoperazione effettua la sottrazione ma scarta il risultato, poiché l'output della ALU non viene memorizzato in alcun registro. Si omettono in questa soluzione i segnali di controllo.

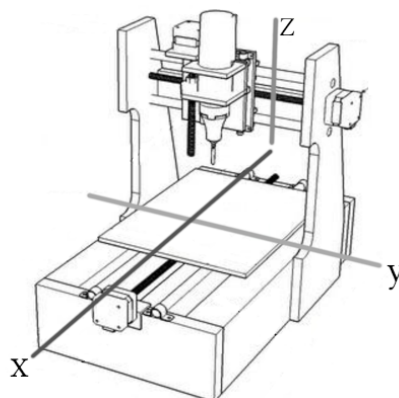
Esercizio 3

Assumendo $x = 0$ e $y = 1$, si calcolano *per divisioni per 2 successive* i valori $A = (102)_{10} = (0000000001100110)_2$ e $B = (314)_{10} = (0000000100111010)_2$. Si noti che i valori sono stati rappresentati, come richiesto, utilizzando 16 bit, anteponendo degli zeri. Essendo i valori positivi ed essendo entrambi $< 2^{15}$, questi *sono già rappresentati in complemento a 2*. La somma è facilmente calcolabile in $(0000000110100000)_2$. Per rappresentare questo valore in esadecimale è possibile raggruppare le cifre in quartetti ed effettuare la conversione verso le cifre esadecimali, ottenendo $(01A0)_{16}$.

Progetto

Un processore z64 comanda una macchina utensile costituita da un **TRAPANO** a colonna e da un **PIANO** su cui viene poggiata un'asse di legno da forare. Il **PIANO** può spostarsi lungo gli assi x e y , per allineare il pezzo di legno al **TRAPANO**. Il **TRAPANO** può muoversi lungo l'asse z per procedere alla foratura. Un **SENSORE** posto sul piano informa lo z64 quando la punta del trapano ha trapassato il pezzo di legno da forare.

SENSORE opera in maniera *asincrona*. **PIANO** e **TRAPANO** operano in modalità *sincrona* e sono dotati di motori passo passo che consentono di effettuare uno spostamento lungo uno degli assi di $1mm$: ogni volta che i dispositivi vengono avviati, essi effettueranno uno spostamento pari a $1mm$. Degli appositi registri di interfaccia consentono di indicare in quale direzione lungo l'asse corrispondente un motore deve effettuare lo spostamento passo passo. Il sistema è schematizzato nella seguente figura:



Un vettore `buchi` memorizzato nella memoria dello z64 individua le posizioni in cui è necessario effettuare i buchi sull'asse. Ciascun elemento del vettore è una word: gli 8 bit più significativi identificano la coordinata x , mentre gli 8 bit meno significativi individuano la coordinata y . Sia la coordinata x che la coordinata y sono degli interi senza segno. All'accensione del sistema, `PIANO` si trova alle coordinate $(0,0)$. Anche `TRAPANO` si trova alla coordinata 0. Poiché `PIANO` può essere posizionato manualmente ad una altezza variabile, è necessario gestire a software il ritorno di `TRAPANO` alla posizione iniziale dopo ciascun buco.

Il vettore `buchi` deve essere configurato per effettuare i buchi nelle posizioni $(3mm, 4mm)$; $(6mm, 8mm)$; $(2mm, 2mm)$.

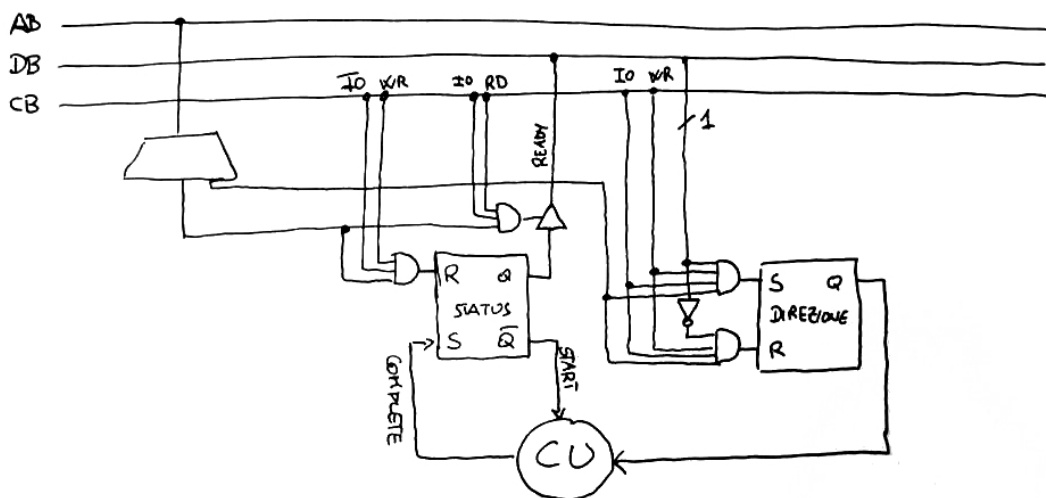
Realizzare:

- Le interfacce dei dispositivi `TRAPANO`, `PIANO` e `SENSORE`;
- Tutto codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

La seguente è una sola delle molte possibili soluzioni dell'esercizio.

La periferica `TRAPANO` è una periferica di *output*, che per poter essere programmata correttamente richiede di sapere se ci si deve spostare verso valori di z crescenti o decrescenti. Per questo scopo, si utilizza un flip flop `DIREZIONE`. Essendo sincrona, opera in modalità busy waiting. Si ottiene pertanto la seguente interfaccia:



La periferica `PIANO` è concettualmente molto simile a `TRAPANO` ma deve avere la possibilità di specificare lungo quale asse il piano deve effettuare un movimento. Poiché la periferica utilizza motori passo-passo, si è deciso di permettere il movimento in direzione x o y , uno per volta. Per questo motivo, si prevede l'utilizzo di due flip flop di `STATUS`, uno per ciascun asse di movimento. Un flip flop `DIREZIONE` permette di specificare, dato un asse, se ci si muove verso valori di posizione crescenti o decrescenti, come nel caso di `TRAPANO`. Si ottiene quindi la seguente interfaccia:


```

20     sti # Per ricevere le interruzioni da SENSORE
21     xorq %r12, %r12 # Contatore del prossimo buco da effettuare
22 .loop:
23     cmpq $num_buchi, %r12
24     jz .end
25     movq %r12, %rdi # Indice del prossimo foro
26     call muovi_piano
27     call trapana
28     addq $1, %r12
29     jmp .loop
30 .end:
31     cli
32     hlt
33
34 muovi_piano:
35     push %rsi
36     movb buchi(%rsi), %dil # recupera la coordinata y (a causa della little endianness)
37     movb posizione + 1, %sil # posizione corrente in y
38     movw $PIANO_STATUSY, %dx # l'asse lungo cui ci si deve muovere
39     call muovi_asse
40
41     movb buchi+1(%rsi), %dil # recupera la coordinata x (a causa della little endianness)
42     movb posizione, %sil # posizione corrente in x
43     movw $PIANO_STATUX, %dx # l'asse lungo cui ci si deve muovere
44     call muovi_asse
45     ret
46
47 # In RDI la coordinata da raggiungere, in RSI la coordinata corrente, in DX l'asse
48 muovi_asse:
49     # Calcola la direzione
50     movb $CRESCENTE, %al
51     subb %sil, %dil # calcola distanza
52     jns .ok
53     movb $DECRESCENTE, %al # la sottrazione è negativa
54     negb %dil # calcola valore assoluto distanza
55 .ok:
56     outb %al, $PIANO_DIREZIONE
57 .loop2:
58     jz .moved # nella prima iterazione, utilizza ZF impostato dalla negb sopra
59     outb %al, %dx # muove lungo l'asse richiesto
60     call do_bw
61     subb $1, %dil
62     jmp .loop2
63 .moved:
64     ret
65
66 trapana:
67     movb $CRESCENTE, %al
68     outb %al, $TRAPANO_DIREZIONE
69     movw $TRAPANO_STATUS, %dx
70 .ciclo_trapano:
71     cmpb $1, trapanato
72     jz .finito
73     outb %al, %dx
74     call do_bw
75     addb $1, movimenti_trapano
76     jmp .ciclo_trapano
77 .finito:
78     movb $DECRESCENTE, %al # Riporta il trapano a posto
79     outb %al, $TRAPANO_DIREZIONE
80 .ciclo_trapano2:
81     outb %al, %dx
82     call do_bw
83     subb $1, movimenti_trapano
84     jnz .ciclo_trapano2
85     movb $0, trapanato

```

```

86     ret
87
88 do_bw:
89     inb %dx, %al
90     btb $0, %al
91     jnc do_bw
92     ret
93
94 .driver 0 # SENSORE
95     movb $1, trapanato
96     outb %al, $SENSORE_IRQ
97     iret

```

Prova di laboratorio

Si vuole realizzare un software per la gestione della programmazione dei film di un cinema, capace di tenere traccia dei film in programmazione nelle diverse sale. Il sistema dovrà permettere le seguenti operazioni:

1. *aggiunta di un film*: l'utente inserisce tutte le informazioni associate al film, permettendone la memorizzazione nel sistema;
2. *ricerca di un film*: l'utente può inserire in input il titolo del film ed il software restituirà tutte le informazioni ad esso associate;
3. *eliminazione di un film*: dato il titolo del film inserito dall'utente, tutte le informazioni legate al film vengono eliminate dalla memoria.

Ogni film è caratterizzato da un titolo, un regista, un anno di uscita e una durata in minuti. Utilizzare una struct denominata `film` per rappresentare ciascun film, includendo i campi `char *titolo`, `char *regista`, `unsigned int anno_uscita` e `unsigned int durata`. Si implementi altresì una struct `programmazione` che contenga un puntatore a un array di puntatori a `film` e un contatore per registrare il numero di film disponibili.

Il numero di film non è noto a priori. È quindi necessario gestire correttamente l'allocazione e la riallocazione dell'array di puntatori a `film`.

La funzione di aggiunta dovrà ricevere i dati del film da inserire e aggiungerlo all'array di programmazione, allocando dinamicamente lo spazio necessario per titolo e regista. La funzione di ricerca restituirà un puntatore al film cercato, se presente, altrimenti NULL. La funzione di cancellazione dovrà rimuovere il film dall'array di programmazione e liberare correttamente la memoria occupata dal film, inclusi i campi dinamici.

Soluzione

La seguente è una possibile soluzione all'esercizio.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  struct film {
6      char *titolo;
7      char *regista;
8      unsigned int anno_uscita;
9      unsigned int durata;
10 };
11

```



```

12 struct programmazione {
13     struct film **elenco_film;
14     unsigned int num_film;
15 };
16
17 static struct film *crea_film(char *titolo, char *regista, unsigned int anno_uscita, unsigned int
durata)
18 {
19     struct film *f = malloc(sizeof(struct film));
20     if (!f) {
21         perror("Errore di allocazione per film");
22         exit(EXIT_FAILURE);
23     }
24     f->titolo = strdup(titolo);
25     f->regista = strdup(regista);
26     f->anno_uscita = anno_uscita;
27     f->durata = durata;
28     return f;
29 }
30
31 static void libera_film(struct film *f)
32 {
33     if (f) {
34         free(f->titolo);
35         free(f->regista);
36         free(f);
37     }
38 }
39
40 static void aggiungi_film(struct programmazione *p, char *titolo, char *regista, unsigned int
anno_uscita, unsigned int durata)
41 {
42     struct film *nuovo_film = crea_film(titolo, regista, anno_uscita, durata);
43     p->elenco_film = realloc(p->elenco_film, (p->num_film + 1) * sizeof(struct film *));
44     if (!p->elenco_film) {
45         perror("Errore di riallocazione per elenco_film");
46         exit(EXIT_FAILURE);
47     }
48     p->elenco_film[p->num_film] = nuovo_film;
49     p->num_film++;
50 }
51
52 static struct film *cerca_film(struct programmazione *p, char *titolo)
53 {
54     for (unsigned int i = 0; i < p->num_film; i++) {
55         if (strcmp(p->elenco_film[i]->titolo, titolo) == 0) {
56             return p->elenco_film[i];
57         }
58     }
59     return NULL;
60 }
61
62 static void elimina_film(struct programmazione *p, char *titolo)
63 {
64     for (unsigned int i = 0; i < p->num_film; i++) {
65         if (strcmp(p->elenco_film[i]->titolo, titolo) == 0) {
66             libera_film(p->elenco_film[i]);
67             for (unsigned int j = i; j < p->num_film - 1; j++) {
68                 p->elenco_film[j] = p->elenco_film[j + 1];
69             }
70             p->num_film--;
71             p->elenco_film = realloc(p->elenco_film, p->num_film * sizeof(struct film *));
72             return;
73         }
74     }
75 }

```

```

76
77 static void leggi_stringa(char *prompt, char *str, size_t max_len)
78 {
79     printf("%s", prompt);
80     if (fgets(str, max_len, stdin) != NULL) {
81         size_t len = strlen(str);
82         if (len > 0 && str[len - 1] == '\n') {
83             str[len - 1] = '\0'; // Rimuove il carattere di newline
84         }
85     } else {
86         fprintf(stderr, "Errore nella lettura della stringa.\n");
87         exit(EXIT_FAILURE);
88     }
89 }
90
91 static unsigned int leggi_intero(char *prompt)
92 {
93     char buffer[50];
94     char *endptr;
95     unsigned long int numero;
96
97     while (1) {
98         printf("%s", prompt);
99         if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
100             numero = strtoul(buffer, &endptr, 10);
101             if (endptr != buffer && *endptr == '\n') {
102                 return (unsigned int)numero;
103             } else {
104                 printf("Input non valido. Per favore, inserisci un numero intero.\n");
105             }
106         } else {
107             fprintf(stderr, "Errore nella lettura dell'input.\n");
108             exit(EXIT_FAILURE);
109         }
110         if (!strchr(buffer, '\n')) {
111             while (getchar() != '\n');
112         }
113     }
114 }
115
116 int main(void)
117 {
118     struct programmazione cinema = {NULL, 0};
119     char scelta, titolo[100], regista[100];
120     unsigned int anno_uscita, durata;
121
122     while (1) {
123         printf("\nMenu di Gestione Cinema:\n");
124         printf("1. Aggiungi un film\n");
125         printf("2. Cerca un film\n");
126         printf("3. Elimina un film\n");
127         printf("Inserisci la tua scelta: ");
128         scelta = getchar();
129         while (getchar() != '\n');
130
131         switch (scelta) {
132             case '1':
133                 leggi_stringa("Inserisci il titolo del film: ", titolo, sizeof(titolo));
134                 leggi_stringa("Inserisci il regista del film: ", regista, sizeof(regista));
135                 anno_uscita = leggi_intero("Inserisci l'anno di uscita: ");
136                 durata = leggi_intero("Inserisci la durata (in minuti): ");
137                 aggiungi_film(&cinema, titolo, regista, anno_uscita, durata);
138                 printf("Film aggiunto con successo.\n");
139                 break;
140             case '2':
141                 leggi_stringa("Inserisci il titolo del film da cercare: ", titolo, sizeof(titolo));

```

```
142         struct film *trovato = cerca_film(&cinema, titolo);
143         if (trovato != NULL) {
144             printf("Film trovato:\n Titolo: %s\n Regista: %s\n Anno di uscita: %u\n Durata:
%u minuti\n",
145                 trovato->titolo, trovato->regista, trovato->anno_uscita, trovato-
>durata);
146         } else {
147             printf("Film non trovato.\n");
148         }
149         break;
150     case '3':
151         leggi_stringa("Inserisci il titolo del film da eliminare: ", titolo,
sizeof(titolo));
152         elimina_film(&cinema, titolo);
153         printf("Film eliminato (se presente).\n");
154         break;
155     default:
156         printf("Scelta non valida. Riprova.\n");
157     }
158 }
159 }
```

